

PBL 05: 단일 EC2 환경의 서비스 분리와 아키텍처 재설계 보고서

정하은

1. 개요

본 보고서는 단일 EC2 인스턴스에 웹(Web) 및 애플리케이션(Application) 서비스가 결합되어 운영되는 현재 아키텍처의 한계를 분석하고, 이를 **마이크로서비스 아키텍처(Microservices Architecture, MSA)**로 전환하여 서비스의 확장성, 유연성, 운영 효율성을 높이는 재설계 방안을 제안하고자 한다. 서비스 간의 결합도를 낮추고 독립적인 운영 환경을 구축함으로써, 변화하는 비즈니스 요구사항에 대응할 수 있는 클라우드 환경을 구축하는 데 중점을 두었다.

2. 문제 정의 및 요구사항

현재 관리자는 하나의 EC2 인스턴스에서 웹 서비스와 애플리케이션 서비스를 함께 제공하는 구조로 인해 여러 가지 운영상의 어려움을 겪고 있다. 이러한 결합된 구조는 다음과 같은 문제점을 야기할 수 있다.

- **낮은 확장성** : 웹 트래픽 또는 애플리케이션 처리량 중 한쪽만 증가하더라도 전체 인스턴스를 확장해야 하므로 리소스 사용이 비효율적이다.
- **낮은 유연성** : 웹 또는 애플리케이션 중 한 부분만 업데이트하더라도 전체 서비스를 재배포하거나 중단해야 할 수 있다.
- **단일 장애점** : 해당 EC2 인스턴스에 장애가 발생하면 웹과 애플리케이션 서비스가 동시에 중단되어 서비스 가용성이 낮다.

이에 따라, 관리자는 EC2 하나에 종속된 웹과 애플리케이션을 분리하여, 마이크로서비스 아키텍처의 운영 장점을 가져가는 환경으로 재설계하기를 원하고 있다.

기존 아키텍처 구성

- VPC 안에 Public Subnet과 Private Subnet 구역이 각각 존재
- Public Subnet 안에 Web과 App이 결합된 하나의 EC2 인스턴스로 위치
- Private Subnet 안에 RDS 위치

3. 마이크로서비스 아키텍처 (MSA) 기본 이해

마이크로서비스 아키텍처는 하나의 거대한 모놀리식 애플리케이션을 작고 독립적인 서비스들의 집합으로 나누는 소프트웨어 개발 방식이다. 각 서비스는 자체적인 프로세스를 가지며, 독립적으로 개발, 배포, 확장, 관리될 수 있다.

주요 특징

- **서비스 분리** : 애플리케이션을 기능 단위(예: 사용자 인증, 주문 처리, 결제 등)로 분리한다. 본 과제에서는 '웹 서비스'와 '애플리케이션 서비스'로 분리하였다.
- **독립적인 배포** : 각 서비스는 다른 서비스와 독립적으로 배포될 수 있으므로, 업데이트 및 릴리스 주기가 단축된다.

- **독립적인 확장** : 특정 서비스의 부하가 증가하면 해당 서비스만 독립적으로 확장할 수 있어 리소스 효율성이 높다.
- **결합도 감소** : 서비스 간의 느슨한 결합(Loose Coupling)으로 인해 한 서비스의 변경이나 장애가 다른 서비스에 미치는 영향이 최소화된다.
- **기술 스택 다양성** : 각 서비스에 가장 적합한 프로그래밍 언어나 데이터베이스를 선택할 수 있다.
- **장점** : 민첩성, 확장성, 유연성, 복원력, 개발 생산성 향상
- **단점** : 분산 시스템의 복잡성 증가, 서비스 간 통신 오버헤드, 데이터 일관성 관리의 어려움

4. 기존 아키텍처 분석 및 결합 해제 가능 부분 식별

주어진 아키텍처는 Public Subnet 내 단일 EC2 인스턴스에 웹(Web)과 애플리케이션(App)이 결합된 형태로 운영되고 있다. 이러한 모놀리식 구조는 다음과 같은 구체적인 문제점을 야기할 수 있다.



- **자원 비효율성** : 웹 서비스는 주로 사용자 인터페이스와 정적 콘텐츠 제공에 중점을 두는 반면, 애플리케이션 서비스는 복잡한 비즈니스 로직 처리와 데이터베이스 연동에 더 많은 컴퓨팅 자원을 요구할 수 있다. 두 서비스가 한 인스턴스에 묶여 있으면, 한쪽의 자원 요구량이 높아질 때 다른 쪽이 불필요하게 확장되거나, 반대로 자원 부족 현상이 발생할 수 있다.
- **배포 및 업데이트의 복잡성** : 웹 UI의 작은 변경이나 애플리케이션 로직의 사소한 수정이라도 전체 인스턴스를 재배포해야 하므로, 배포 시간이 길어지고 서비스 중단 위험이 커진다.
- **장애 전파 가능성** : 웹 또는 애플리케이션 중 한 부분에서 발생하는 오류가 전체 인스턴스에 영향을 미쳐 서비스 전체의 중단으로 이어질 수 있다.

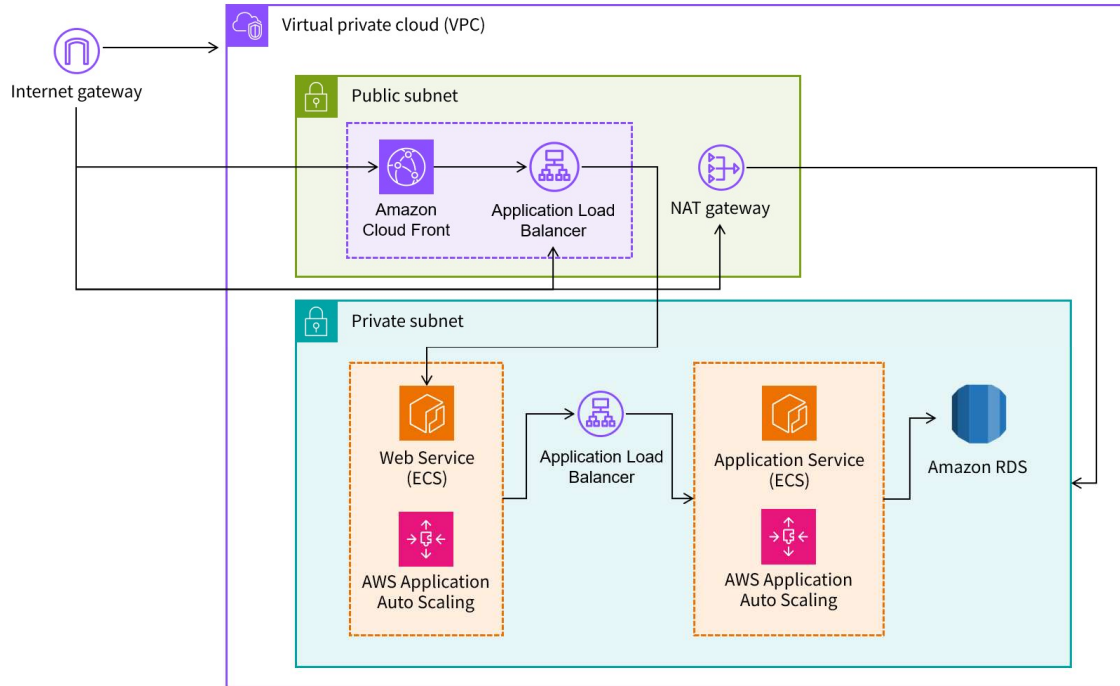
이러한 문제점을 해결하기 위해, 다음과 같이 논리적으로 분리 가능한 두 가지 핵심 부분을 식별했다.

- **웹 서비스 (Web Service)** : 사용자에게 직접 노출되는 프론트엔드 계층으로, UI 렌더링, 사용자 입력 처리, 정적 콘텐츠 제공 등을 담당한다.
- **애플리케이션 서비스 (Application Service)** : 백엔드 계층으로, 웹 서비스의 요청을 받아 복잡한 비즈니스 로직을 처리하고 데이터베이스와 상호작용한다.

이 두 서비스를 독립적인 마이크로서비스로 분리하여 운영하는 것이 재설계의 핵심이다.

5. 재설계된 아키텍처 제안 및 마이크로서비스 전환

기존 아키텍처의 웹과 애플리케이션 결합 구조를 해제하고, AWS 관리형 서비스를 활용하여 마이크로서비스 아키텍처의 장점을 극대화하는 방향으로 재설계했다. 특히 컨테이너 서비스(Amazon ECS Fargate)를 활용하여 각 서비스를 독립적으로 배포하고 확장할 수 있도록 구성했다.



5.1. 웹 서비스 분리 및 배포

- **목표** : 사용자에게 직접 노출되는 웹 서비스를 독립적인 계층으로 분리하고, 높은 가용성과 확장성을 확보하도록 했다.
- **선택 서비스**
 - **Application Load Balancer (ALB)** : Public Subnet에 배치되어 외부 사용자 트래픽을 받아 Private Subnet 내의 웹 서비스로 분산한다. HTTP/HTTPS 트래픽을 효율적으로 처리하고, 경로 기반 라우팅 등을 지원한다.
 - **Amazon ECS (Elastic Container Service) with Fargate** : 웹 애플리케이션을 컨테이너화하여 ECS Fargate에 배포한다. Fargate는 서버리스 컨테이너 컴퓨팅 엔진으로, 사용자가 EC2 인스턴스(컨테이너 호스트)를 직접 관리할 필요 없이 컨테이너를 실행할 수 있게 하여 관리 부담을 대폭 줄일 수 있다.
 - **설명** : ALB는 인터넷 게이트웨이를 통해 들어오는 사용자 요청을 받아 Private Subnet에 위치한 웹 서비스 컨테이너(ECS Fargate)로 전달한다. ECS Fargate는 웹 서비스의 트래픽 변화에 따라 자동으로 컨테이너 인스턴스를 확장/축소하여 유연한 대응이 가능하며, 이는 웹 서비스의 독립적인 확장을 가능하게 한다.

5.2. 애플리케이션 서비스 분리 및 배포

- **목표** : 웹 서비스로부터 분리된 애플리케이션 비즈니스 로직을 독립적인 서비스로 운영하고, 데이터베이스와의 효율적인 통신을 보장한다.
- **선택 서비스**
 - **Internal Application Load Balancer (Internal ALB)** : Private Subnet에 배치되어 웹 서비스로부터 들어오는 내부 요청을 받아 애플리케이션 서비스로 분산한다. 웹 서비스와 애플리케이션 서비스 간의 내부 통신을 효율적으로 관리하고, 애플리케이션 서비스의 고가용성을 확보한다.
 - **Amazon ECS (Elastic Container Service) with Fargate** : 애플리케이션 비즈니스 로직을 컨테이너화하여 ECS Fargate에 배포한다. 웹 서비스와 마찬가지로, Fargate를 통해 서버 관리 없이 애플리케이션 서비스를 독립적으로 확장하고 배포할 수 있다.
- **설명** : 웹 서비스 컨테이너는 내부 ALB를 통해 애플리케이션 서비스 컨테이너(ECS Fargate)와 통신한다. 애플리케이션 서비스는 비즈니스 로직을 처리하고, 필요에 따라 데이터베이스(Amazon RDS)와 상호작용한다. 각 애플리케이션 서비스는 독립적으로 확장될 수 있어, 특정 비즈니스 로직의 부하가 증가해도 전체 시스템에 영향을 주지 않는다.

5.3. 데이터베이스 (Amazon RDS)

- **유지** : 기존 아키텍처에서 이미 Private Subnet에 배치된 Amazon RDS (MySQL)는 관리형 데이터베이스 서비스이므로, 별도의 관리 부담이 적다. 따라서 재설계된 아키텍처에서도 그대로 유지하며, 애플리케이션 서비스와 안전하게 통신하도록 하였다.

5.4. 기타 네트워크 및 보안 구성

- **NAT Gateway** : Private Subnet 내의 웹 및 애플리케이션 서비스 컨테이너가 외부 인터넷(예: 소프트웨어 업데이트, 외부 API 호출)으로 나갈 필요가 있을 경우, Public Subnet에 위치한 NAT Gateway를 통해 안전하게 통신한다.
- **보안 그룹 (Security Groups)** : 각 서비스 계층(ALB, 웹 서비스, 애플리케이션 서비스, RDS)에 적절한 보안 그룹을 적용하여 최소 권한 원칙에 따라 트래픽을 제어하고 보안을 강화한다.

6. 결론

본 재설계는 단일 EC2 인스턴스에 결합된 웹 및 애플리케이션 환경을 AWS의 관리형 서비스(ALB, ECS Fargate, Internal ALB, RDS)를 활용하여 마이크로서비스 아키텍처로 성공적으로 전환하는 방안을 제시한다. 이러한 전환을 통해 다음과 같은 운영 장점을 확보할 수 있을 것이다.

- **향상된 확장성** : 웹 서비스와 애플리케이션 서비스를 독립적으로 확장할 수 있어, 리소스 사용 효율성이 극대화되고 트래픽 변화에 유연하게 대응할 수 있습니다.
- **높은 유연성 및 민첩성** : 각 서비스가 독립적으로 배포되므로, 특정 서비스의 업데이트나 변경이 다른 서비스에 미치는 영향을 최소화하고, 개발 및 배포 주기를 단축할 수 있습니다.
- **강화된 복원력** : 한 서비스에 장애가 발생하더라도 다른 서비스에 미치는 영향이 제한되어 전체 시스템의 가용성이 향상됩니다.
- **관리 부담 감소** : ECS Fargate와 RDS와 같은 관리형 서비스를 통해 인프라 관리 및 운영 오버헤드를 AWS에 위임하여, 관리자는 핵심 비즈니스 로직 개발에 집중할 수 있습니다.

결론적으로, 마이크로서비스 아키텍처로의 전환은 서비스의 운영 효율성, 안정성, 확장성, 그리고 개발 민첩성을 향상시켜, 현대 클라우드 환경에서 요구되는 유연하고 견고한 시스템을 구축하는 데 필요한 전략임을 보여준다.